Лабораторна робота №2

Варіант 1 all_of

Треба було дослідити швидкодію sall_of за наданих умов.

Процес

Згенеровуємо 4 файли з числами від 1 до 1000: на 100 тис, 1 млн, 10 млн і 100 млн елементів. Предикат — x > 0, щоб all_of завжди був true і сканував усе.
Компілятор MSVC 2022, C++20, x64. Збирав у Release з /Od і /O2. hardware_concurrency = 12.
Паралельний all_of: ділимо вектор на K частин, кожну обробляємо all_of у своєму jthread, потім зводимо результати теж через all_of.

Результати

Бібліотечний all_of **(**/O2**), час у секундах:**

N	No policy	seq	par	unseq	Par unseq
100 тис	0.000027	0.000120	0.000174	0.000124	0.000090
1 млн	0.000332	0.001220	0.000433	0.001526	0.000372
10 млн	0.003283	0.012754	0.001919	0.013525	0.002157
100 млн	0.009179	0.032880	0.003867	0.041034	0.004244

Паралельний all_of**, N = 100 000 000 (**/O2**):**

K	Час, с
1	0.008134
2	0.008840
4	0.008032
8	0.004752
16	**0.003575**
32	0.003817

K	Час, с
64	0.005074

Висновки

1. На малих N паралелізм тільки шкодить. На 100 тисячах par програє seq. Потоки дорожче створити, ніж вони встигають щось зробити. Починаючи з мільйона par уже виграє.
2. par і par_unseq дають прискорення 6–8 разів на великих N. На 100 мільйонах par швидший у 8.5 раза. При 12 ядрах це очікувано.
3. unseq взагалі не допомагає. На 10 мільйонах навіть повільніший за seq. Мабуть, через те що all_of має break. Його погано векторизувати SIMD-ом.
4. no policy набагато швидший за seq. На 100 мільйона у 3.6 раза. MSVC без явної політики використовує якийсь свій швидкий шлях, а seq то вже узагальнений повільніший варіант.
5. Best K = 16, а ядер 12. Тобто найкраща кількість потоків більша за кількість ядер. Це через те, що задача memory-bound. x > 0 це одна операція, а от число з пам'яті тягнеться довго. Тому більше потоків краще перекривають затримки RAM.
6. При K > best час зростає приблизно лінійно. Це вже overhead від створення jthread і перемикань контексту.
7. Мій алгоритм на великих N навіть трохи обігнав бібліотечний par. Напевно, бо я вибираю оптимальне K, а бібліотека має фіксовану стратегію.
8. Рівень оптимізації майже ні на що не впливає. /Od і /O2 дають однакові числа в межах похибки, іноді /O2 навіть повільніший. Це тому що задача впирається в пам'ять, а не в CPU. Оптимізатор не може прискорити RAM, а предикат x > 0 і так виконується за один такт.